

proton mixing & weight normalization issues

05/02/2026

1. Overview

Goal

Search for exclusive di-tau production in the muon-tau channel using 2018 data with the Precision Proton Spectrometer (PPS).

Samples Used

Sample	Description	Source
Data	2018 UL SingleMuon	Real collision data
DY	Drell-Yan Z/γ^* to tau-tau	MC simulation
ttbar	tt to tau-tau + X	MC simulation
QCD	Multi-jet background	Data-driven (same-sign)

2. Processing Pipeline

NanoAOD --> Phase0 --> Phase1 --> Merge --> Proton Mixing --> Plots

Scripts

Stage	Data	DY	ttbar	QCD
Phase0	fase0_data_mutau.py	fase0_dy.py	fase0.py	-
Phase1	fase1_data.py	fase1_dy.py	fase1_ttjets.py	fase1_qcd.py

3. Phase0: initial skimming

Scripts

- `fase0_data_mutau.py` (Data)
- `fase0_dy.py` (Drell-Yan)
- `fase0.py` (ttbar)

Structure of scripts

- 1 **Trigger selection:** HLT_IsoMu24 (single muon trigger)
- 2 **Luminosity filtering:** Only certified good runs (`dadosluminosidade.txt`)
- 3 **Object selection:** At least 1 muon and 1 tau
- 4 **Variable extraction:** Muon, Tau, Jets, MET kinematics

3. Phase0 (continued)

Proton variables saved (data only)

For MC samples these are added with proton mixing

```
nproton_multi, nproton_single  
proton_multi_xi, proton_multi_arm, proton_multi_t  
proton_multi_thetaX, proton_multi_thetaY  
proton_multi_time, proton_multi_timeUnc  
proton_single_xi
```

Output

- ROOT files with tree containing selected events
- weight = 1.0, generator_weight = 1.0 (for Data)

4. Phase1: Selection cuts

Variable	Cut	Description
muon_id	> 3	Medium MVA ID
tau_id1 (VSjet)	> 63	VeryTight DeepTau vs jets
tau_id2 (VSe)	> 7	Tight DeepTau vs electrons
tau_id3 (VSmu)	> 1	Loose DeepTau vs muons
muon_pt	$> 35 \text{ GeV}$	muon transverse momentum
tau_pt	$> 100 \text{ GeV}$	tau transverse momentum
charge product	< 0	opposite sign
Delta R	> 0.4	Angular separation

Note: QCD uses **same-sign** (SS) selection

5. Event weights in Phase1

Data

```
event_weight = 1.0  # Real data no correction needed
```

Drell-Yan (DY)

```
event_weight = generator_weight x mu_trig_sf x mu_idiso_sf x mu_reco_sf
```

ttbar

```
event_weight = 0.15 x mu_trig_sf x mu_idiso_sf x mu_reco_sf
```

Important: 0.15 already included - should be fixed later for consistency

QCD

```
event_weight = 1.0  # data driven
```

6. Proton Pool

Collection of real protons from 2018 PPS data, used to simulate pileup protons in MC.

Location

`/eos/cms/store/group/phys_smp/Exclusive_DiTau/proton_pool_2018/`

Contents

Branch	Description
<code>proton_arm</code>	detector arm
<code>proton_xi</code>	fractional momentum loss

How it was created

- 1 Start with 2018 data with signal triggers
- 2 No kinematic cuts on the central system
- 3 Extract protons from PPS

7. Proton Mixing Algorithm

MC samples (DY, ttbar) don't have reliable pileup protons - they need to be added.

Script: `merge_pp_mutau.py`

Step 1: Always assign 1 proton per arm

- 1 proton with `arm=0` -> stored as `xi_arm1_1`
- 1 proton with `arm=1` -> stored as `xi_arm2_1`

Step 2: Probabilistically add 2nd protons

Config	Probability	Description
P11	8.0%	1 proton on arm0, 1 proton on arm1
P12	2.0%	1 proton on arm0, 2 protons on arm1
P21	2.0%	2 protons on arm0, 1 proton on arm1
P22	0.5%	2 protons on each arm

8. Proton Acceptance

The probability that a random event has **at least 1 proton detected on both arms** simultaneously.

Why?

- After proton mixing: 100% of MC events have 1+ protons per arm
- This does not reflect real data
- We use a weight to account for events without pileup protons

8. Proton Acceptance (measurement)

Sample: Data_2018_UL_MuTau_nano_merged_proton_vars.root

Category	Events	Fraction
No protons on either arm	1,849	26.5%
Protons only on arm 0	1,600	23.0%
Protons only on arm 1	1,813	26.0%
Protons on both arms	1,708	24.5%

Total events: 6,970

My measurement: $P = 0.245$ (24.5%)

Matteo's original: $P = 0.13$ (13%)

Needed: which value is correct for this analysis?

9. Bug in code - pileup proton mixing

Original Code

```
# In merge_pp_mutau.py:  
weight = array("d", [float(FIXED_WEIGHT)])  
t_out.Branch("weight", weight, "weight/D")  
# In event loop: weight was never updated -> hard coded to 0.13  
Problem: The original event_weight was discarded
```

Fixed code

```
# reads input event_weight if it exists  
if event_weight_in is not None:  
    weight[0] = event_weight_in[0] * FIXED_WEIGHT  
else:  
    weight[0] = FIXED_WEIGHT  
Now: weight = event_weight x 0.13 (preserves original weights)
```

10. Where event weights are applied

Data (no proton mixing needed)

Phase0: $\text{weight}=1.0 \rightarrow$ Phase1: $\text{weight}=1.0 \rightarrow$ Plotting: 1.0

Drell-Yan

Phase1: $\text{event_weight} = \text{generator_weight} \times \text{muon_SFs}$

Proton Mixing: $\text{weight} = \text{event_weight} \times 0.13$

Plotting: $\text{final_weight} = \text{weight} \times 1.81$

10. (cont)

ttbar

Phase1: `event_weight = 0.15 x muon_SF`s <- 0.15 included here

Proton Mixing: `weight = event_weight x 0.13`

Plotting: `final_weight = weight x 1.0`

QCD (data-driven)

Phase1: `weight=1.0` -> Plotting: `final_weight=1.0`

11. Cross-Section normalization factors

Formula used

$\text{weight} = N_{\text{MC}} / N_{\text{expected}}$
where $N_{\text{expected}} = \text{Luminosity} \times \text{Cross-section}$

Values used

Sample	Factor	Included Where?
DY	1.81	Applied in plotting
ttbar	0.15	Already in Phase 1 event_weight

This does not change the outcome but should be fixed for consistency

12. Status of plotting code

File: plot_m.py

How weights are currently used:

```
# QCD: fixed weight
w_qcd = 1.0
# DY: reads weight from tree (currently contains only 0.13)
w_dy = get_value(event, "weight", default=1.81)
# ttbar: reads weight from tree (currently contains only 0.13)
w_ttjets = get_value(event, "weight", default=0.15)
```

Issues:

- 1 DY and ttbar files have `weight = 0.13` (`event_weight` lost)
- 2 DY needs additional 1.81 factor not being applied
- 3 ttbar has 0.15 in `event_weight` but it's lost in merged files

13. Summary of issues found

Issue	Status	Action Required
proton mixing overwrites weights	code fixed	re-run proton mixing
DY cross-section factor missing	pending	apply 1.81 in plotting
proton acceptance value unclear	pending	decide: 0.13 or 0.245
ttbar event_weight lost	code fixed	re-run proton mixing

14. Next steps

Essential

- 1 Decide on proton acceptance value (0.13 or 0.245)
- 2 **Re-run proton mixing** for DY and ttbar with fixed script
- 3 **Update plotting code** to apply correct normalizations

Verification

- 4 **Generate comparison plots** before/after fix

15. Files

Processing scripts

Script	Purpose
fase0_data_mutau.py	Phase0 for Data
fase0_dy.py	Phase0 for DY
fase1_data.py	Phase1 for Data
fase1_dy.py	Phase1 for DY
fase1_ttjets.py	Phase1 for ttbar
merge_pp_mutau.py	Proton mixing
plot_m.py	Plotting

16. Proton acceptance measurement code

Method in ROOT:

```
TFile* f = TFile::Open("Data_merged_proton_vars.root");
TTree* tree = (TTree*)f->Get("tree");
Long64_t total = tree->GetEntries();
Long64_t good = tree->GetEntries(
    "Sum$(proton_multi_arm==0)>=1 && Sum$(proton_multi_arm==1)>=1"
);
double P = (double)good / (double)total;
cout << "Proton acceptance P = " << P << endl;
```

Result:

```
total events: 6,970
good events: 1,708
P = 0.245 (24.5%)
```

17.proton mixing validation:multiplicity

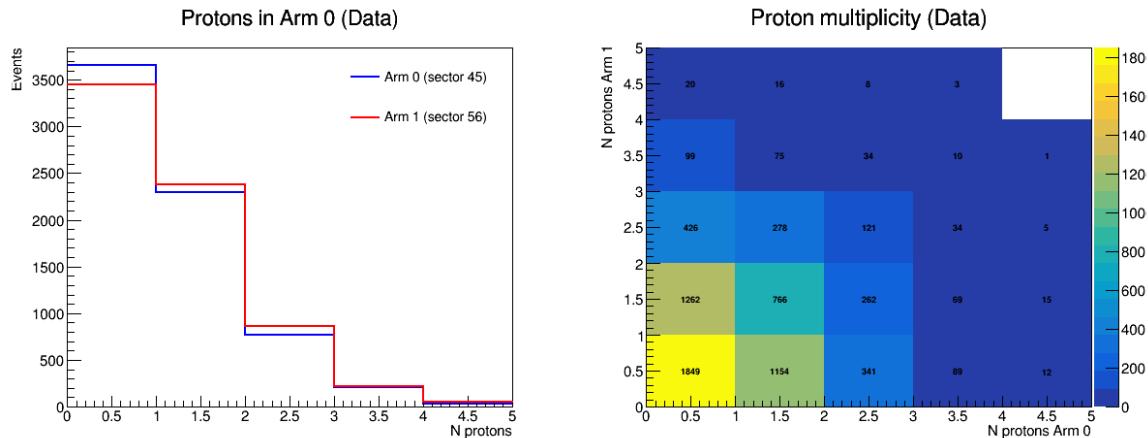


Figure 1: Proton multiplicity in data

- Left: Number of protons per arm in data

18.xi Distributions

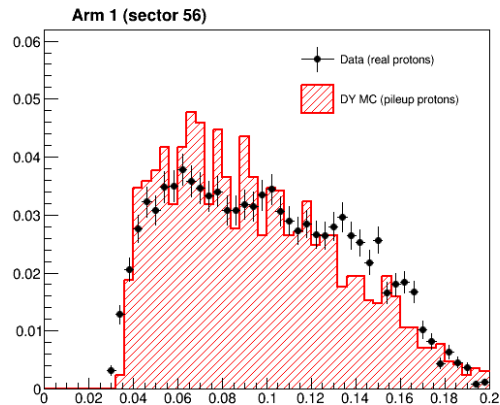
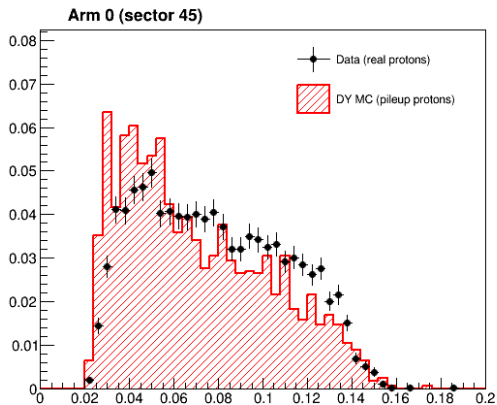
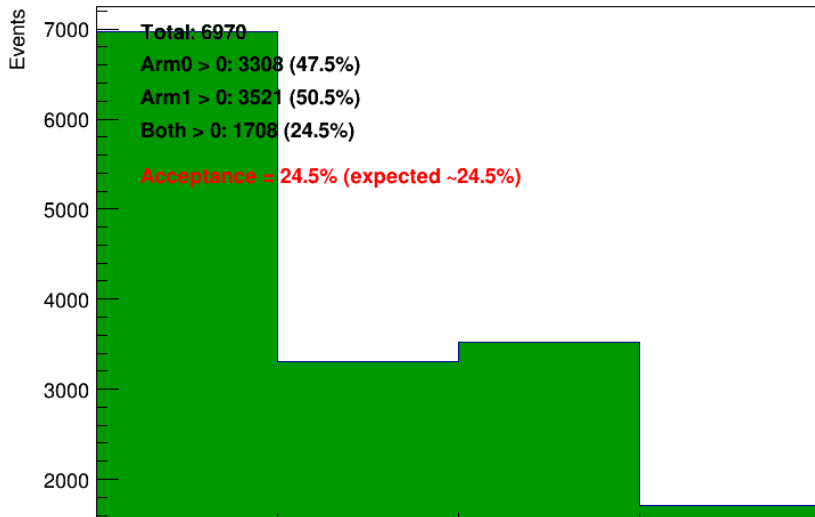


Figure 2: ξ comparison

- Comparison of ξ distributions: Data vs MC

Data Proton Check



22. Comparison Plots: Before vs After Proton Mixing

Variables plotted

Variable	Description
sist_mass	Invariant mass $m(\mu, \tau)$
acop	Acoplanarity 1 -
sist_pt	System transverse momentum
sist_rap	System rapidity
tau_pt	Tau transverse momentum
met_pt	Missing transverse energy

Comparison setup

- **Left:** Before proton mixing (no proton requirements)
- **Right:** After proton mixing (require protons on both arms)

23. Comparison Plots (continued)

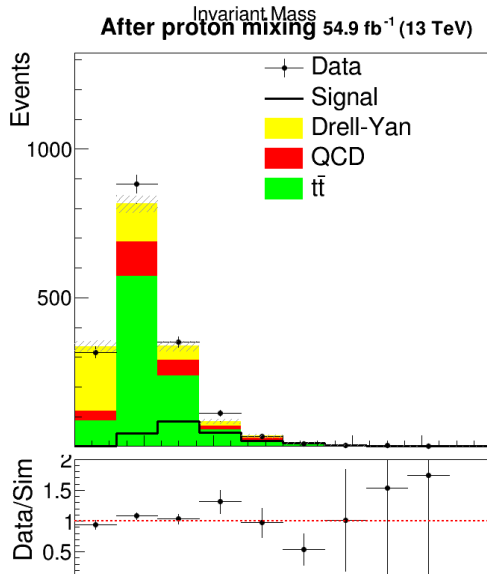
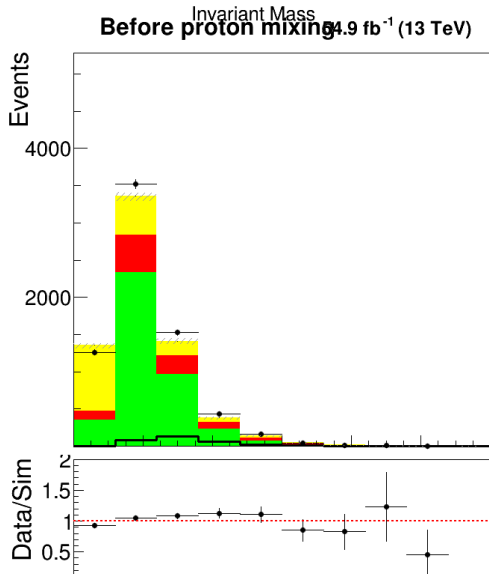
Files used

Sample	Before mixing	After mixing
Data	<code>_proton_vars.root</code>	<code>_proton_vars.root + proton cut</code>
QCD	<code>_proton_vars.root</code>	<code>_proton_vars.root + proton cut</code>
DY	<code>_merged.root</code>	<code>_pileup_protons.root</code>
ttbar	<code>_merged.root</code>	<code>_pileup_protons.root</code>

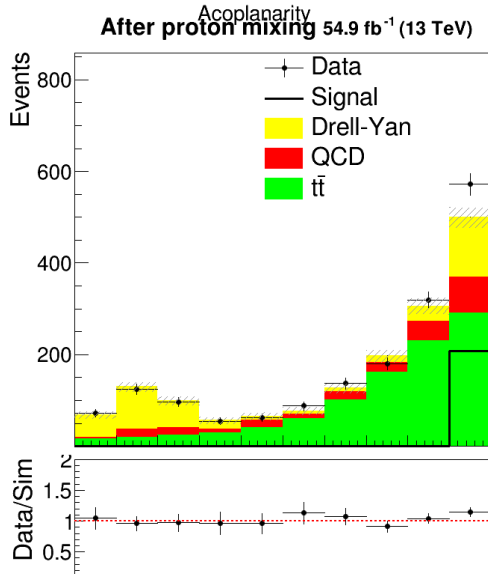
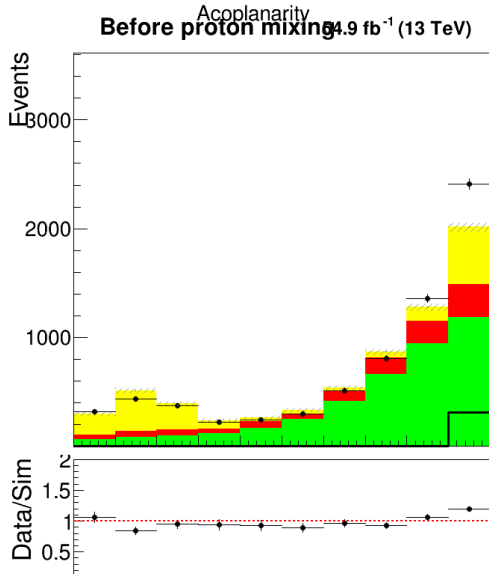
Proton requirements

- **Data/QCD:** `Sum$(proton_multi_arm==0)>0 && Sum$(proton_multi_arm==1)>0`
- **MC (mixed):** `xi_arm1_1 >= 0 && xi_arm2_1 >= 0`
- **MC weight:** Scaled by proton acceptance (0.245) after mixing

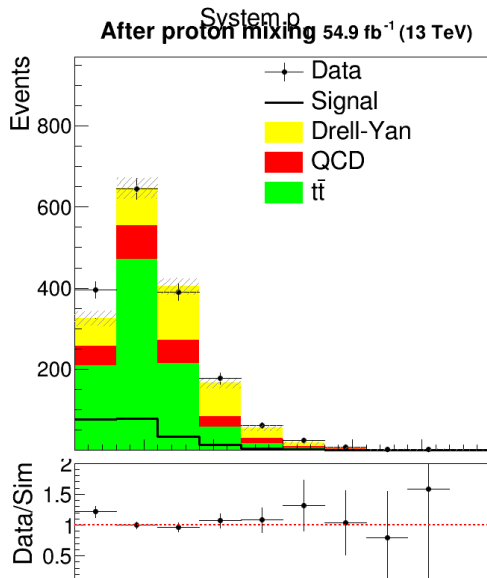
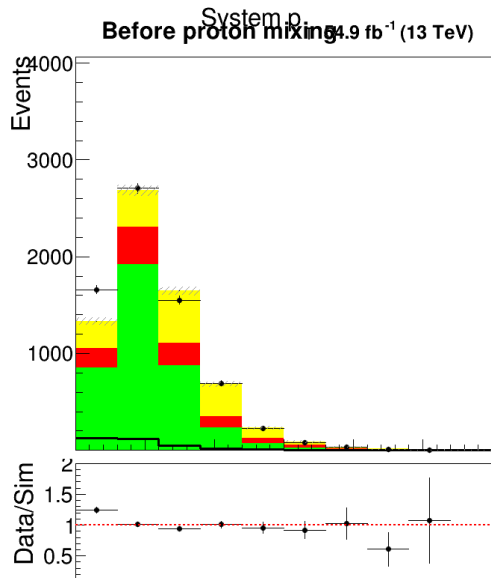
24. Invariant Mass Comparison



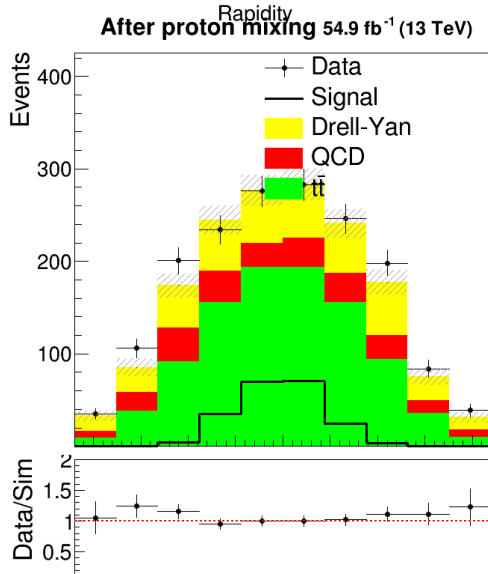
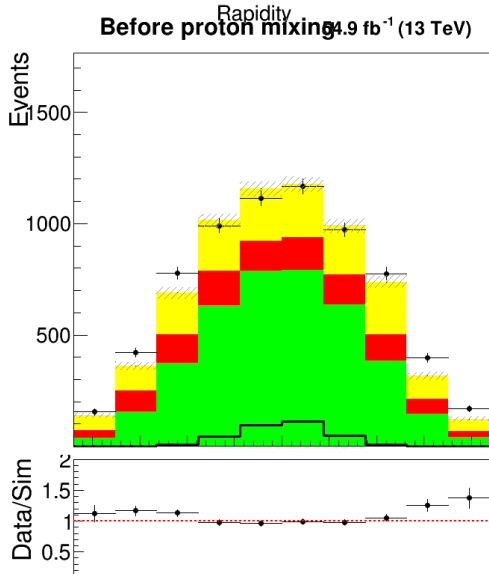
25. Acoplanarity Comparison



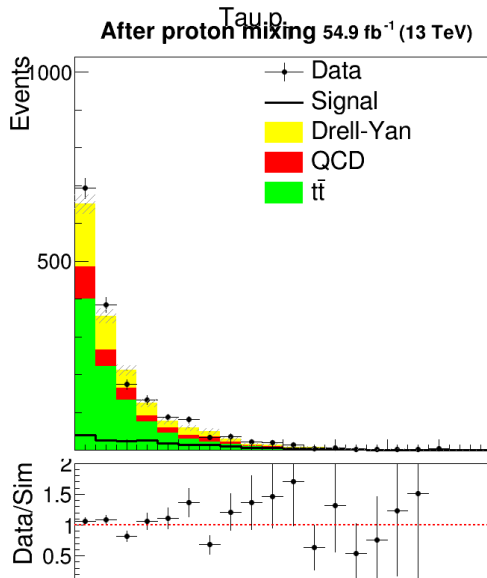
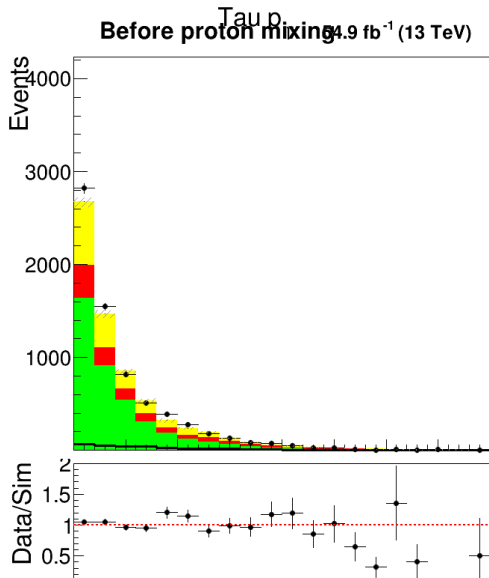
26. System pT Comparison



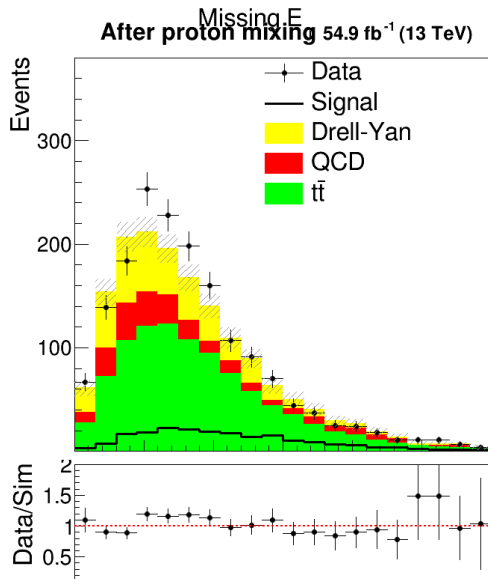
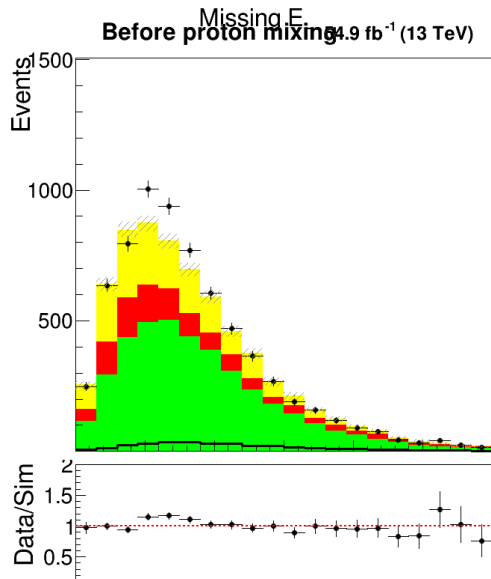
27. Rapidity Comparison



28. Tau pT Comparison



29. MET Comparison



30. Derived Variables (MC only)

Rapidity matching

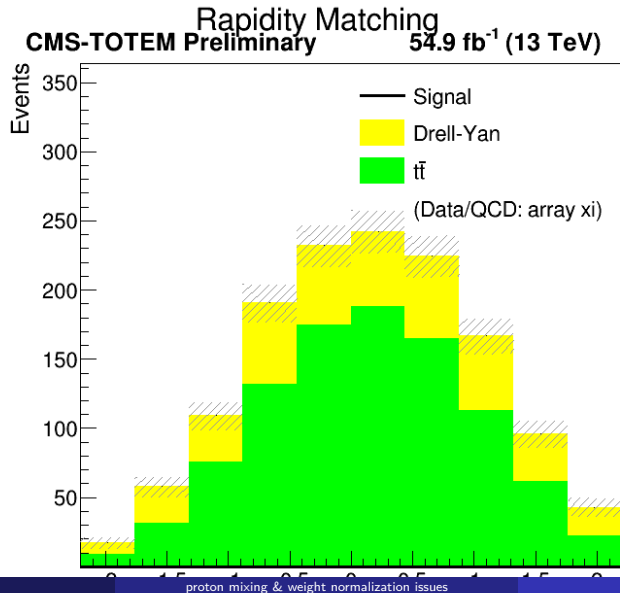
$$y_{\text{match}} = y(\mu, \tau) - 0.5 * \log(x_{i1} / x_{i2})$$

Mass difference

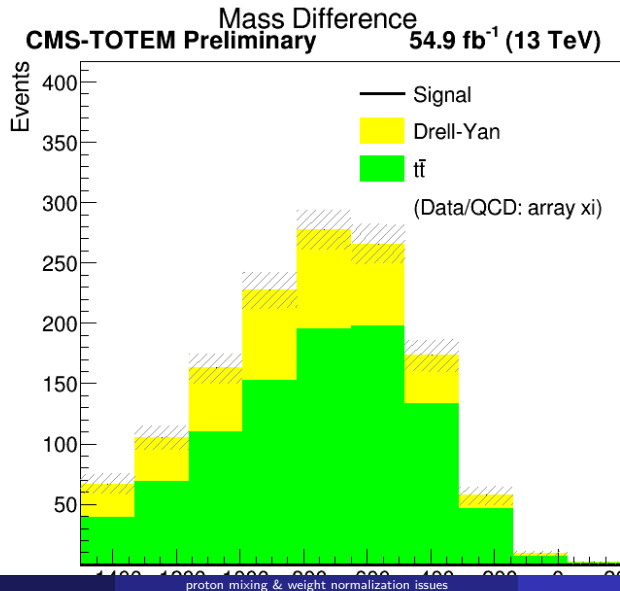
$$m_{\text{diff}} = m(\mu, \tau) - 13000 * \sqrt{x_{i1} * x_{i2}}$$

- these require xi branches, available only in MC/Signal files

31. Rapidity Matching Plot



32. Mass Difference Plot



33. Single Lepton Distributions

Muon variables

- `mu_pt` - Muon transverse momentum
- `mu_eta` - Muon pseudorapidity
- `mu_mass` - Muon mass

Tau variables

- `tau_pt` - Tau transverse momentum
- `tau_eta` - Tau pseudorapidity
- `tau_mass` - Tau visible mass

33b. Single Lepton Plots: Cuts Applied

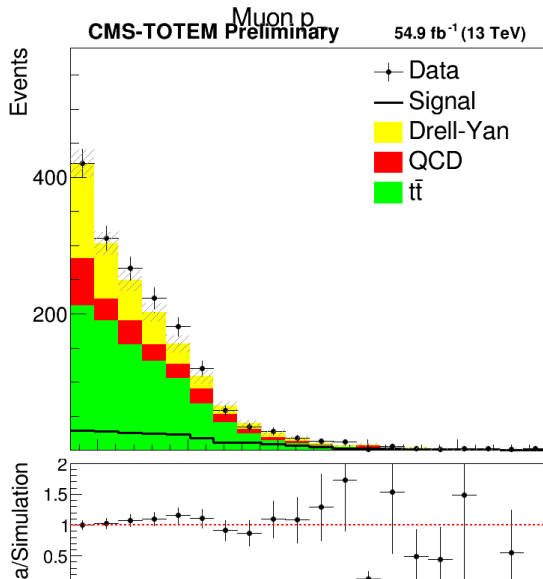
Proton requirements

Sample	Cut
Data/QCD	<code>Sum\$(proton_multi_arm==0)>0 && Sum\$(proton_multi_arm==1)>0</code>
MC (DY, ttbar)	<code>xi_arm1_1 >= 0 && xi_arm2_1 >= 0</code>
Signal	<code>xi_arm1_1 >= 0 && xi_arm2_1 >= 0</code>

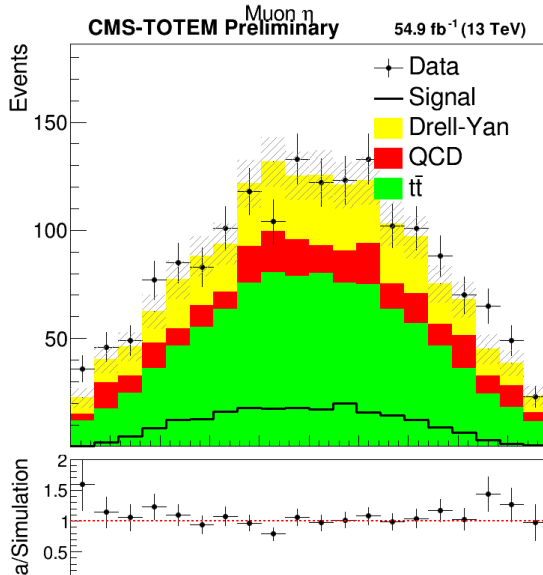
Weights applied

Sample	Weight
Data	None
QCD	None
DY	Proton acceptance (0.245)
ttbar	Proton acceptance (0.245) $\times$ Scale(0.15)
Signal	weight branch (lumi $\times$ xsec $\times$ SFs $\times$ rad. damage)

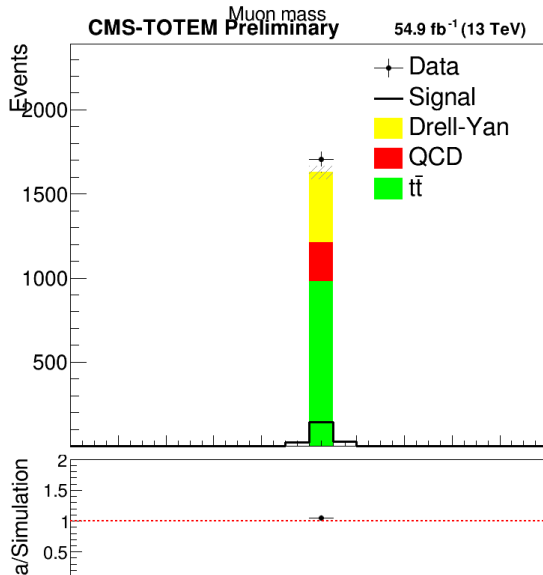
34. Muon pT



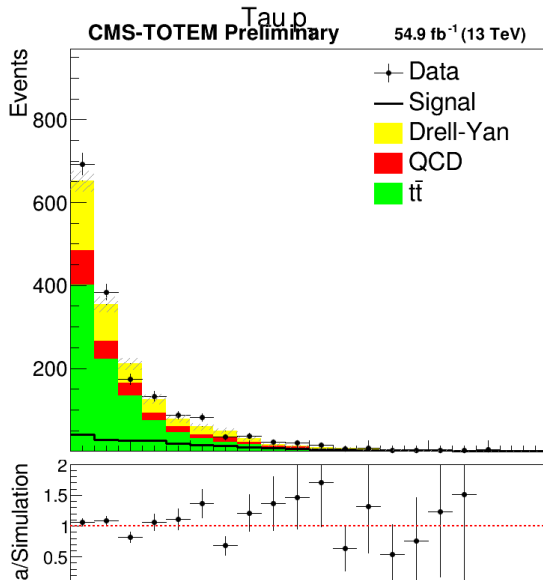
35. Muon eta



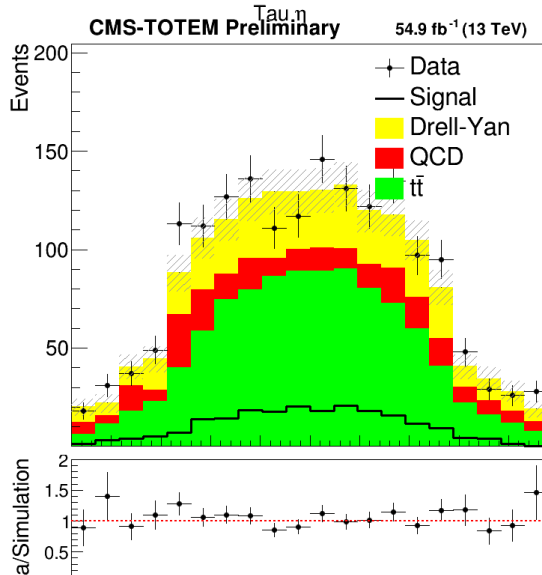
36. Muon mass



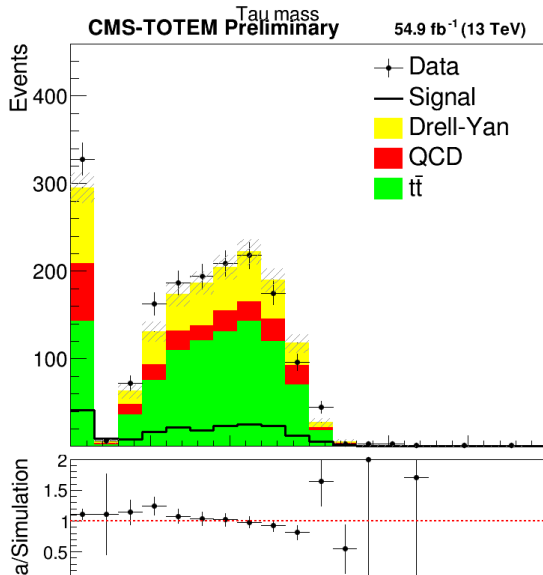
37. Tau pT



38. Tau eta



39. Tau mass



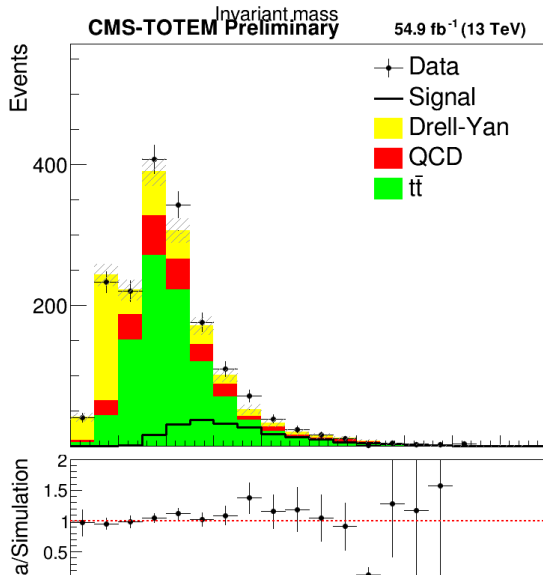
40. Single Distributions After Proton Merging

The following plots show individual kinematic distributions after requiring protons on both PPS arms.

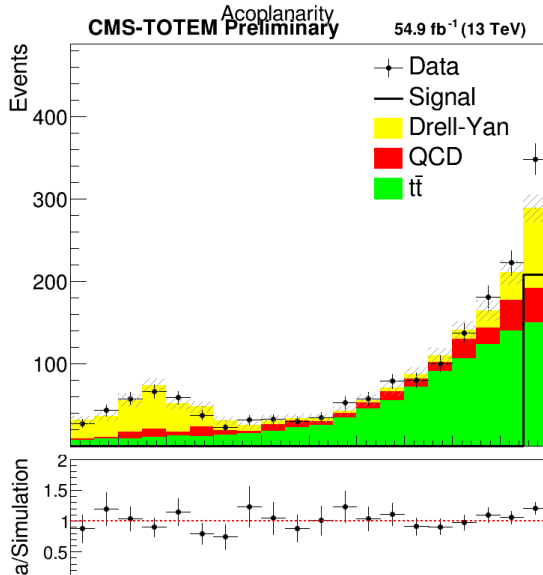
Variables shown

Variable	Description
sist_mass	Invariant mass $m(\mu, \tau)$
acop	Acoplanarity
sist_pt	System transverse momentum
sist_rap	System rapidity
tau_pt	Tau transverse momentum
met_pt	Missing transverse energy

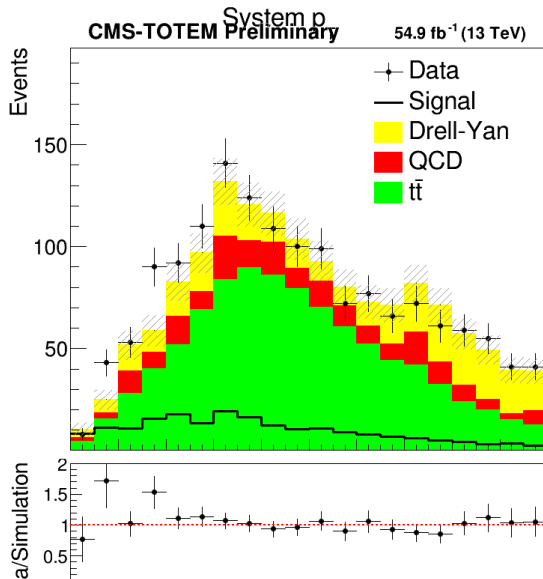
41. Invariant Mass (after proton requirements)



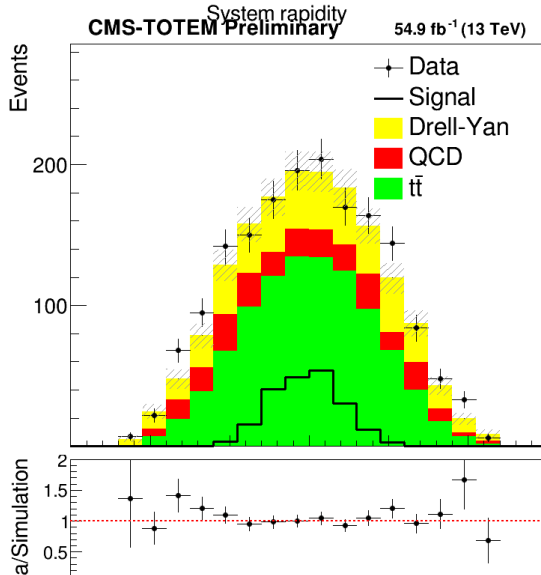
42. Acoplanarity (after proton requirements)



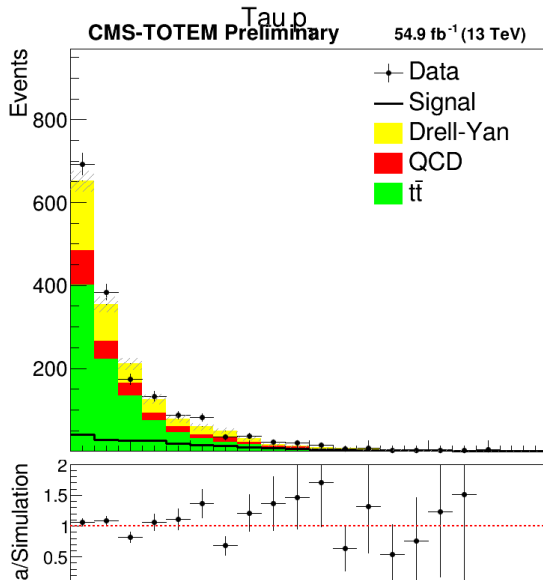
43. System p_T (after proton requirements)



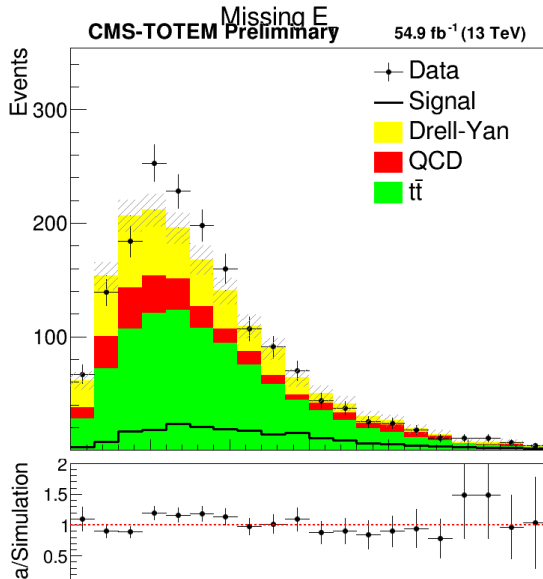
44. System Rapidity (after proton requirements)



45. Tau pT (after proton requirements)



46. MET (after proton requirements)



47. Signal processing logic: sinal.py

- Process the exclusive gamma-gamma $\rightarrow$ tau-tau signal MC sample for the mu-tau channel.

Input

`GammaGammaTauTau_2018_UL_MuTau_SMandBSMweights_ntuplesfromminiJuly.root`

Output

`MuTau_sinal_SM_2018_july.root`

48. sinal.py: SFs Corrections

Muon Scale Factors Applied

SF Type	Function	Description
Trigger	<code>muon_trig_sf()</code>	IsoMu24 trigger efficiency
ID+Iso	<code>muon_idiso_sf()</code>	Identification and isolation
Reco	<code>mu_reco_sf()</code>	Reconstruction efficiency

Tables loaded from POGCorrections/

- `muon_neg18_lowTrig.txt` - Negative muon, $p_T < 200$ GeV
- `muon_pos18_lowTrig.txt` - Positive muon, $p_T < 200$ GeV
- `muon_18_highTrig.txt` - High p_T muons (> 200 GeV)
- `muon_IdISO.txt` - ID and isolation corrections

49. sinal.py: Event Selection cuts applied

Cut	Requirement
Basic	At least 1 muon and 1 tau
ID	$\text{tau_id_full} > 0.5$ and $\text{mu_id} > 0.5$
Eta	$\text{tau_eta} < 2.4$ and $\text{mu_eta} < 2.4$
Delta R	Angular separation > 0.4
pT	$\text{tau_pt} > 100 \text{ GeV}$, $\text{mu_pt} > 35 \text{ GeV}$
Charge	Opposite sign (product < 0)
Protons	Exactly 2 reconstructed protons

50. sinal.py: proton acceptance cuts and apperture cuts (θ_x)

Why crossing angle matters

Signal MC is generated without specific crossing angle, so we must emulate real data conditions.

Crossing angle assignment

Events are assigned a crossing angle based on their position in the file ($\text{fraction} = \text{event_index} / \text{total_events}$), matching the 2018 luminosity profile

50b. sinal.py:apertureCut (theta_x)

- LHC aperture limits proton acceptance

aperture function

The maximum allowed theta_x depends on both **crossing angle** and **xi**:

```
limit_fun_arm_0 = f(xangle, xi)  # linear function  
limit_fun_arm_1 = f(xangle, xi)  # diff for each arm
```

Cut applied

```
if thx[0] > -limit_fun_arm_0.Eval(0.5):  
    continue  # Proton 0 outside aperture  
if thx[1] > -limit_fun_arm_1.Eval(0.5):  
    continue  # Proton 1 outside aperture
```

Events with protons outside the LHC aperture are rejected.

51. sinal.py: fiducial cuts

Track position variables

Each proton has::

- trackx1, tracky1 - Position at near station (RP 210)
- trackx2, tracky2 - Position at far station (RP 220)
- Index [0] = arm 45, Index [1] = arm 56

Period dependence

Detector positions and beam conditions changed during 2018, requiring different fiducial cuts per period.

52. sinal.py: radiation damage corrections

Pixel efficiency

Two efficiency applied per event:

① Multi-RP

- pixelEfficiencies_multiRP_reMiniAOD.root
- Accounts for track matching between stations

② Radiation damage

- pixelEfficiencies_radiation_reMiniAOD.root
- Accounts for sensor degradation over time

Weight calculation

```
weight *= raddamage45.GetBinContent(...) # arm 45  
weight *= raddamage56.GetBinContent(...) # arm 56
```

53. sinal.py: event weight

Final event weight formula

$$\text{weight} = \text{weight_sample} \times \text{mu_trig} \times \text{mu_idiso} \times \text{mu_reco} \times \text{rad_damage}$$

Where:

- $\text{weight_sample} = 54900 \times \text{weight_sm} / (4000 \times 1000)$
 - 54900 fb^{-1} = integrated luminosity
 - weight_sm = generator-level SM weight
 - 4000×1000 = generated events normalization

BSM weights

- 102 BSM weights stored per event
- Normalized to SM cross-section: $\text{bsm_weights}[i] / \text{bsm_weights}[51]$

54. sinal.py: Xi systematics

Proton xi systematic uncertainties

Polynomial fits to systematic uncertainty vs xi:

```
xi_sist_1 = proton_sist_1.Get("../g_systematics_vs_xi")
xi_sist_inter_1 = TF1("xi_sist_inter_1", "pol20", 0, 10)
xi_sist_1.Fit(xi_sist_inter_1)
```

Stored variations

Branch	Description
xi_arm1_1_up	xi + systematic shift
xi_arm1_1_dw	xi - systematic shift
xi_arm2_1_up	xi + systematic shift
xi_arm2_1_dw	xi - systematic shift

55. sinal.py: variables

Kinematic

- Muon: pt, eta, phi, mass, charge, energy, ID
- Tau: pt, eta, phi, mass, charge, energy, ID (full, antimu, antie, antij), decay mode
- System: mass, pt, rapidity, acoplanarity
- MET: pt, phi
- Jets: pt, eta, phi, mass, btag, n_b_jet

Proton variables

- xi per arm (1st and 2nd proton)
- theta_x, theta_y, x, y, t
- Track positions and angles
- Timing information

